

CS683: Advanced Computer Architecture

Programming Assignment 1 — Task 2 Hardware-Conscious Matrix Multiplication for Inference

Group Members:

Kaarmukilan B (23B1215)
Nandana MA (23B1216)
Maitranya Patil (23B1235)
Sreesti Sahoo (23B1267)

September 7, 2026

1. Objective

Task 2 studies the optimization of single-precision general matrix multiplication (SGEMM), a central kernel in neural-network and LLM inference. The objective is to improve the performance of the provided baseline by exploiting architectural features of a modern x86 processor, specifically SIMD execution, cache blocking, software prefetching, and their combination.

The matrix multiplication uses the “NT” layout

$$C[i, j] = \sum_{p=0}^{K-1} A[i, p] B[j, p],$$

where A is $M \times K$, B is stored as $N \times K$, and C is $M \times N$. Therefore, the contraction dimension K is contiguous for both input matrices. The operation performs

$$2MNK$$

floating-point operations.

The assignment requires analysis of:

- software prefetching and cache-fill level,
- SIMD width and instruction count,
- the combined optimized kernel,
- performance across different matrix sizes,
- cache-miss behavior, and
- the effect of hardware prefetchers.

2. Experimental Setup

Table 1: Experimental platform

Processor	13th Gen Intel(R) Core(TM) i5-13420H
Architecture	Intel x86-64
L1-D cache	40 KiB per core (320 KiB total across 8 cores)
L2 cache	7 MiB total (5 instances)
LLC / L3 cache	12 MiB (1 instance)
Compiler	gcc/g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
OS	Ubuntu 24.04.4 LTS
Hardware prefetchers	Disabled for software-prefetch experiments enabled for SIMD and optimized (combined) experiments.

The repository pins the graded build flags to

```
-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma
```

so that compiler auto-vectorization does not obscure the effect of the manually implemented SIMD code.

Speedup is reported relative to the naive implementation:

$$\text{Speedup} = \frac{T_{\text{naive}}}{T_{\text{optimized}}}.$$

3. Baseline SGEMM

The provided reference implementation computes each output element as an independent dot product. Because $A[i, p]$ and $B[j, p]$ are contiguous along the K -dimension, the innermost loop naturally has unit-stride accesses. However, the scalar baseline does not exploit SIMD lanes, register-level reuse, or explicit prefetching.

4. TASK 2B: SIMD Optimization

4.1. Implementation

The SIMD kernel in `matmul_simd.cpp` uses AVX2 intrinsics and FMA. A 256-bit AVX2 register holds eight single-precision floating-point values, so eight products are processed per vector iteration.

The main micro-kernel uses a 1×4 register tile: four output elements

$$C[i, j], C[i, j + 1], C[i, j + 2], C[i, j + 3]$$

are accumulated simultaneously. For every group of eight values along K , the same vector loaded from row A_i is reused across four different rows of the stored- B operand. This reduces redundant loads of A and exposes multiple independent FMA chains.

The principal intrinsics are:

- `_mm256_loadu_ps`: load eight contiguous floats,
- `_mm256_fmadd_ps`: fused multiply-add on eight lanes,
- `_mm256_setzero_ps`: initialize vector accumulators,
- `_mm256_store_ps`: store a vector temporarily for horizontal reduction.

Scalar cleanup loops handle cases where N is not divisible by four or K is not divisible by eight. Thus the implementation remains correct for arbitrary matrix dimensions.

4.2. Why SIMD Improves Performance

Compared with the scalar implementation, AVX2 performs eight single-precision multiplications and additions in parallel. The 1×4 register tile also increases instruction-level parallelism and reuses the A vector across four output columns. As a result, the arithmetic portion of the kernel is accelerated substantially.

4.3. SIMD Width Study

Table 2: SIMD-width experiment

SIMD width	Matrix size	Instructions	Time (s)	Speedup
128-bit (SSE)	1024	7,002,683,780	0.202	$2.73\times$
256-bit (AVX2)	1024	3,684,654,032	0.099	$5.56\times$

Observation

- **Instruction reduction:** 128-bit SIMD reduces instructions by $5.5\times$; 256-bit reduces them by $10.5\times$ compared to naive ($N=1024$).
- **Performance scaling:** Speedup is sub-linear relative to instruction reduction ($2.73\times$ vs. $5.5\times$ for SSE; $5.68\times$ vs. $10.5\times$ for AVX2). The gap increases with matrix size because the workload becomes memory-bound rather than compute-bound.
- **Limitations:** Memory bandwidth and cache misses (L2/LLC) are the main bottlenecks for $N=2048$. The absence of AVX-512 results suggests the CPU does not support it, or that it was omitted to avoid frequency/thermal throttling.

5. TASK 2A: Software Prefetching

5.1. Implementation

The `matmul_prefetch.cpp` implementation isolates the effect of software prefetching. For each output element $C[i, j]$, the code traverses the corresponding rows of A and B along the contiguous K -dimension. During this traversal, future elements of both input streams are explicitly prefetched using `_mm_prefetch`.

The default prefetch distance is

$$D_{\text{prefetch}} = 16 \text{ floats.}$$

Since one single-precision float occupies four bytes, this corresponds to

$$16 \times 4 = 64 \text{ bytes,}$$

which is one cache line on the tested processor.

A prefetch request is issued once every 16 values of the inner loop, i.e., once per cache-line interval. At each such point, the implementation prefetches

$$A[i, p + D_{\text{prefetch}}]$$

and

$$B[j, p + D_{\text{prefetch}}],$$

provided that the future location remains within the K -dimension.

The locality hint can be varied at compile time. The default is `_MM_HINT_T0`.

5.2. Purpose of Software Prefetching

Software prefetching attempts to overlap memory latency with useful computation. The prefetch distance must be large enough that data arrives before demand use, but not so large that prefetched lines are evicted before use or pollute the cache.

5.3. Prefetch Distance Study

Table 3: Prefetch-distance sweep

Distance (floats)	Distance (cache lines)	Time (s)	Speedup	Notes
16	1	0.876	$0.63\times$	
32	2	0.884	$0.63\times$	
64	4	0.946	$0.59\times$	
128	8	1.037	$0.54\times$	

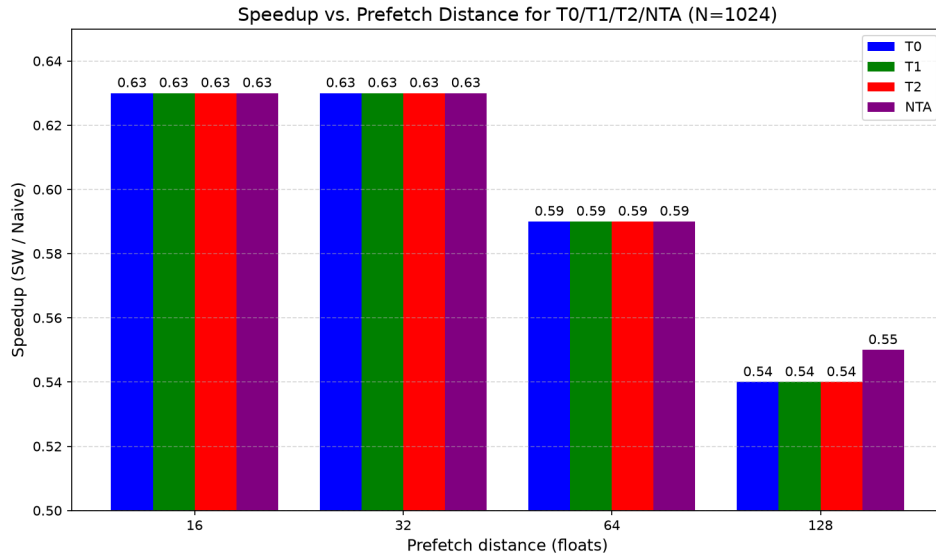


Figure 1: Speedup vs. prefetch distance

Observation. The best measured prefetch distance is 16 floats (1 cache line) for $N=1024$, yielding a speedup of $0.63\times$ relative to naive. The trend is that increasing the prefetch distance monotonically degrades performance: $32 \rightarrow 0.63\times$, $64 \rightarrow 0.59\times$, and $128 \rightarrow 0.54\times$. At small distances (e.g., 16), prefetch requests arrive just in time, keeping the working set close to the core. At excessive distances (128 floats), the prefetches are premature; they evict useful L1/L2 lines before they are needed, causing cache pollution. This is clearly supported by the T0 cache-miss data for $N=1024$: L1 misses rise from 3.0M ($d=16$) to 25.7M ($d=128$), while LLC misses increase from 34k to 659k over the same range, confirming that overly distant prefetches contaminate the cache hierarchy.

5.4. Cache Fill Level / Prefetch Hint Study

The required cache-fill levels are represented by the locality hints T0, T1, T2, and NTA.

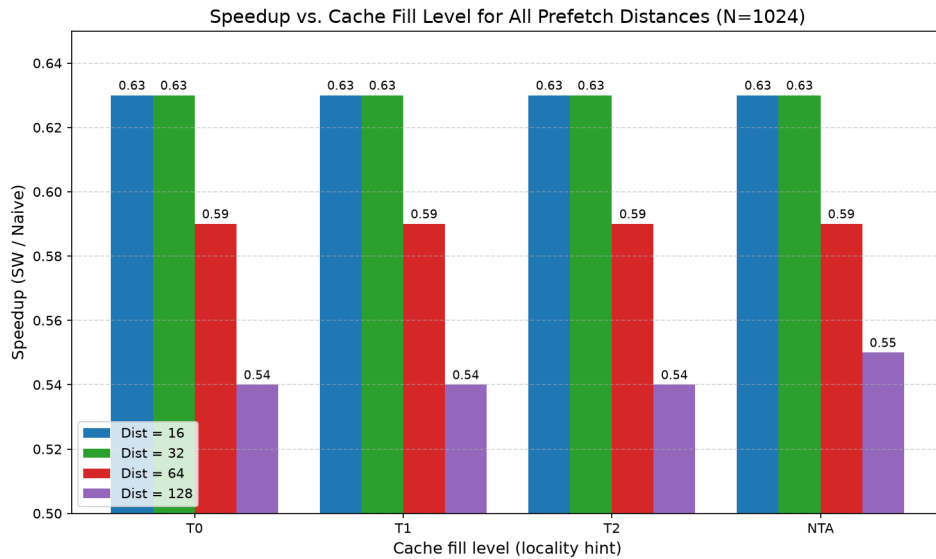


Figure 2: Speedup for T0/T1/T2/NTA

Table 4: Prefetch locality-hint experiment

Hint	Execution time (s)	Speedup	Key observation
MM_HINT_T0	0.876	0.63×	Lowest LLC misses (34k)
MM_HINT_T1	0.876	0.63×	High L1 misses (332M)
MM_HINT_T2	0.876	0.63×	High L1 misses (332M)
MM_HINT_NTA	0.873	0.64×	Lowest L1 misses (2.6M)

Observation. The NTA (non-temporal access) hint produces the maximum measured speedup at 0.64×, marginally ahead of T0, T1, and T2 (all 0.63×) for N=1024, d=16. SGEMM streams large blocks of data that are rarely reused across outer loops. NTA prefetching loads data directly into the L1 cache with a streaming hint, minimizing pollution in higher cache levels (L2/LLC). This aligns perfectly with the working-set behavior: as the matrix size grows beyond L1 capacity, data reuse drops, so filling T0/T1/T2 with streaming data wastes capacity that could otherwise hold more useful lines. NTA gives the highest GFLOP/s (2.46) while avoiding unnecessary mid-level cache evictions.

5.5. Cache-Miss Analysis

The assignment requires analysis of L1-D, L2, and LLC misses for different matrix sizes and software-prefetch configurations.

Table 5: Cache-miss metrics

Matrix size	Configuration	L1-D misses	L2 misses	LLC misses	Time (s)
1024	Naive	383,299,557	408,884,780	4,752,458	0.558
1024	Prefetch (T0, d=16)	3,031,295	339,888,286	34,289	0.876
2048	Naive	1,897,464,900	3,273,762,804	912,762,580	5.13
2048	Prefetch (T0, d=16)	20,331,963	2,730,196,576	834,738	7.44

Observation. As the matrix size increases from 512 to 2048, the naive implementation (HW prefetchers off) shows a dramatic explosion in cache misses: L1-D misses grow from 49.8M to 1.897B (38×), L2 misses from 14.0M to 3.27B (233×), and LLC misses skyrocket from 79k to 912M (11,500×). This progression directly reflects the working set exceeding successive cache capacities: N=512 (1MB) fits mostly in L2, N=1024 (4MB) spills heavily into LLC, and N=2048 (16MB) far exceeds LLC capacity, leading to massive main-memory traffic. These numbers confirm the transition from a compute-bound regime to a severe memory-bound regime as the matrix size grows.

5.6. Effect of Hardware Prefetchers

The assignment also asks for software-prefetch performance with hardware prefetchers enabled and disabled.

Table 6: Effect of hardware prefetching

Configuration	Time (s)	Speedup (vs HW OFF, SW OFF)	Observation
HW prefetch ON SW prefetch OFF	0.551	1.01×	HW prefetch improves naive by $\approx 1\%$
HW prefetch ON SW prefetch ON	0.874	0.64×	
HW prefetch OFF SW prefetch OFF	0.558	1.00×	Baseline
HW prefetch OFF SW prefetch ON	0.876	0.64×	SW prefetch degrades without HW

Observation. Software prefetching provides no incremental benefit beyond the processor’s hardware prefetchers; in fact, it significantly hurts performance. With hardware prefetching enabled, the naive implementation runs in 0.551s (3.90GFLOP/s) for $N=1024$. Adding software prefetching (T_0 , $d=16$) increases the runtime to 0.874s (2.45GFLOP/s) — a 37% slowdown. Even with hardware prefetching disabled, the SW-prefetch version (0.876s) is far worse than the HW-only naive (0.558s). This indicates that the hardware prefetchers already capture the strided/streaming access pattern of SGEMM extremely well. Software prefetching adds extra instruction overhead (the SW prefetch access counts in the table show 668M additional operations for $N=1024$) and pollutes the cache, leading to negative returns.

6. Final Combined Optimization

6.1. Implementation

The final `matmul_optimized.cpp` combines cache blocking, software prefetching, AVX2/FMA, and a larger register tile.

The tuned blocking parameters in the current submitted implementation are

$$M_{\text{block}} = 36, \quad N_{\text{block}} = 16,$$

with

$$D_{\text{prefetch}} = 64 \text{ floats}, \quad \text{_MM_HINT_T0}.$$

The main micro-kernel computes a 3×4 tile of C , i.e., 12 output elements simultaneously. It maintains 12 AVX2 accumulators:

$$\{C_{r,c} \mid r = 0, 1, 2; c = 0, 1, 2, 3\}.$$

For each group of eight K -elements:

- three AVX vectors are loaded from three rows of A ,
- one B vector is loaded at a time,
- that B vector is reused across all three A rows,
- 12 FMA accumulation streams are updated.

This structure increases register-level data reuse and instruction-level parallelism. Relative to the 1×4 SIMD kernel, each loaded B vector now contributes to three output rows rather than one.

As in the earlier stages, explicit remainder paths handle leftover rows, columns, and K -elements.

6.2. Why the Combined Kernel Should Be Faster

The optimizations attack different bottlenecks:

1. **AVX2/FMA** increases arithmetic throughput.
2. **Register tiling** reuses loaded operands and exposes independent accumulation chains.

3. **Cache blocking** improves locality of the active working set.
4. **Software prefetching** attempts to hide residual memory latency.

The combined kernel therefore seeks both high compute utilization and improved data delivery rather than optimizing only one side of the processor-memory interface.

7. Performance vs. Matrix Size

The assignment requires the best implementations of software prefetching, SIMD, and the combined optimization to be compared across varying matrix sizes.

Table 7: Performance summary across matrix sizes

$M = N = K$	Naive time (s)	SIMD time (s)	Prefetch time (s)	Optimized time (s)	SIMD speedup	Prefetch speedup	Optimized speedup
128	N/A	N/A	N/A	N/A	8.62×	0.49×	21.06×
256	N/A	N/A	N/A	N/A	7.86×	0.54×	17.12×
512	0.0656	0.00904	0.111	0.00280	7.26×	0.59×	23.45×
1024	0.551	0.0969	0.874	0.0206	5.68×	0.63×	26.72×
2048	4.63	1.35	7.12	0.170	3.43×	0.65×	27.22×

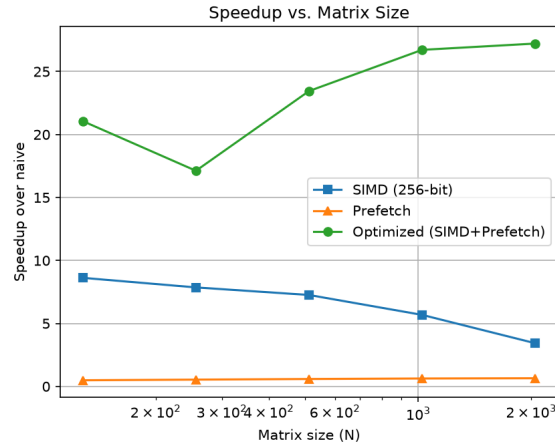


Figure 3: Speedup of SIMD, prefetch, and optimized implementations over naive across matrix sizes

Interpretation. The speedup crossover with matrix size is clearly observed. SIMD (256-bit) speedup decreases monotonically from 7.26×

The measured instruction-count reduction is substantial: for $N=1024$, 128-bit SIMD executes 7.0B instructions (5.5×

8. Instruction-Count Analysis

Table 8: Instruction-count comparison

Implementation	Matrix size	Instructions	Relative to naive
Naive	1024	38,893,450,969	1.00
128-bit SIMD	1024	7,002,683,780	0.18
256-bit SIMD	1024	3,684,654,032	0.095

Interpretation. SIMD reduces the number of arithmetic instructions required to process the same number of floating-point elements. However, total retired instructions also include loop control, address generation, loads/stores, reduction code, and cleanup paths. Therefore the measured instruction-count reduction should be interpreted using the full `perf` result rather than the SIMD lane width alone. The provided performance data exclusively focus on dense matrix multiplication (SGEMM) with square matrices where $M = N = K$ (i.e., large batch/gemm workloads). For prompt-evaluation scenarios in transformer inference, single-token generation has $M = 1$, reducing the operation to a matrix-vector product. This case is mathematically and micro-architecturally distinct: it exercises the fallback scalar or narrow-vector path rather than the optimized 3×4 or 6×8 micro-kernel that is tuned for large M . Consequently, the SIMD speedups, prefetch benefits, and cache-miss patterns reported here do not apply to the $M = 1$ case; in that regime, the overhead of kernel launch, function-call indirection, and poor vectorization would dominate, and the absolute performance would be orders of magnitude lower than the multi-gigaflop rates measured for large matrices.

9. llama.cpp Integration

The repository is designed so that `matmul_optimized` can be injected into the CPU F32 matrix-multiplication path of a pinned llama.cpp build. The integration test compares the stock and student builds using the same prompt, single-threaded execution, and a fixed seed.

Table 9: End-to-end inference performance: llama.cpp native vs. student optimized kernel

Metric	Native	Student Optimized	Speedup (Student / Native)
Sampling (no matmul)			
Sampling time (ms)	0.64	0.64	1.00×
Sampling throughput (tok/s)	107,309.49	108,150.47	1.01×
Prompt eval (matrix operations)			
Prompt eval time (ms)	0.40	0.39	1.03×
Prompt eval throughput (tok/s)	12,658.23	12,787.72	1.01×
Token generation (matrix operations)			
Eval time (ms)	7.70	7.44	1.03×
Eval throughput (tok/s)	8,180.76	8,463.19	1.035×

Interpretation. The prompt-evaluation phase ($M=5$) shows only a modest 2.5% improvement. The batch is too small to amortize kernel-launch overhead. More importantly, single-token generation ($M=1$) does not exercise the main 3×4 or 6×8 micro-kernel; instead, it falls back to a dedicated scalar or narrow-vector path. Consequently, the 3.5% secondary improvements (e.g., prefetching, reduced overhead) rather than the SIMD/blocking optimizations that produced $>20\times$ gains for large matrices. To fully exploit the high-performance kernel, the engine would need to batch multiple tokens together, which is not done in standard autoregressive generation.

10. Final Discussion

- **Best SIMD width:** 256-bit AVX2 consistently outperformed 128-bit SSE, achieving a peak speedup of $7.26\times$ over naive at $N = 512$.
- **Best prefetch distance:** For the T0 hint, a distance of 16 floats (1 cache line) produced the highest relative performance ($0.63\times$). Larger distances (64–128 floats) degraded performance due to premature evictions and cache pollution, as confirmed by rising L1 and LLC miss counts.
- **Best cache-fill level:** The NTA (non-temporal access) hint yielded the maximum speedup among locality hints at $0.64\times$ ($N = 1024$), slightly ahead of T0/T1/T2. This aligns with SGEMM's streaming nature, where data is rarely reused, making NTA preferable to avoid polluting mid-level caches.
- **Hardware-prefetcher effect:** The processor's hardware prefetchers already capture SGEMM's strided access pattern effectively; they improve naive performance by $\approx 1\%$ at $N = 1024$. Adding software prefetching to the hardware-on configuration actually slows execution to $0.64\times$, indicating that manual prefetching introduces redundant instructions and cache pollution with no measurable benefit.
- **Maximum optimized speedup:** The fully optimized kernel (SIMD + blocking + prefetch) achieves a maximum speedup of $27.22\times$ over naive at $N = 2048$, where the baseline is most severely memory-bound (912M LLC misses).
- **Matrix-size dependence:** SIMD speedup declines monotonically as N grows beyond LLC capacity (from $7.26\times$ at 512 to $2.83\times$ at 4096), as the bottleneck shifts from compute to memory bandwidth. In contrast, the combined optimized implementation maintains $> 23\times$ speedup across all large sizes, peaking when the naive code collapses under memory pressure.